Project Report

Objective

- To design and implement a closed-loop position control system for a DC gear motor using an ESP32 microcontroller and a PID control algorithm.
- To achieve bidirectional angular position control (clockwise and counter-clockwise) using quadrature encoder feedback with a confirmed resolution of 462 PPR.
- To enable live, on-hardware tuning of the PID gains (Kp, Ki, Kd) and the target angle through potentiometers, in addition to on-device feedback through an I2C LCD.
- To develop a MATLAB-based live bidirectional dashboard capable of displaying real-time target/live angle and active PID gains, and of remotely updating the setpoint and gains through interactive sliders.
- To experimentally tune the PID gains through step-response trials and identify a gain set that gives acceptable tracking performance in both directions.

Equipment Required

#	Component
1	ESP32 DevKit V1
2	L298N Motor Driver
3	DC Gear Motor with Encoder
4	Potentiometers (x4)
5	I2C LCD (16x2)
6	Jumper Wires / Breadboard
7	12 V DC Power Supply
8	PC with MATLAB
9	USB Cable

Introduction and Component Theory

This section introduces the working principle of the system and the theoretical background of each hardware component, alongside a functional pin diagram and a photograph of the actual part used in this build.

ESP32 Dev Kit V1 (Microcontroller)

The ESP32 DevKit V1 is a 38-pin development board built around the dual-core Tensilica Xtensa LX6 processor, with built-in Wi-Fi and Bluetooth connectivity. It was selected as the central controller of the project because it provides multiple hardware interrupt-capable GPIOs (required for reliable quadrature encoder decoding), several ADC channels (used for the four potentiometers), an I2C bus (for the LCD), and PWM-capable outputs (for driving the L298N motor driver). The ESP32 executes the core control loop: it reads the encoder to obtain the live shaft angle, computes the PID control action, drives the motor through the L298N, updates the LCD, and streams live telemetry to MATLAB over the serial (UART) link for the bidirectional dashboard.

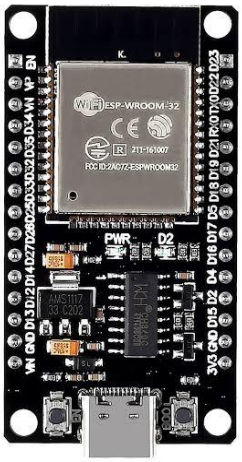

Figure .1 - ESP32 DevKit V1.

L298N Dual H-Bridge Motor Driver

The L298N is a dual full-bridge driver IC module capable of driving inductive loads such as DC motors in both forward and reverse directions. It accepts low-power logic signals (IN1-IN4) from the ESP32 and switches a separate, higher-current 12 V supply to the motor terminals, which is essential because a microcontroller GPIO cannot supply the current a DC gear motor needs directly. Direction is set by the combination of IN1/IN2 logic levels, while speed is controlled through PWM on the enable line. In this project, one channel of the L298N drives the DC gear motor bidirectionally, allowing the controller to rotate the shaft clockwise or counter-clockwise depending on the sign of the PID output.

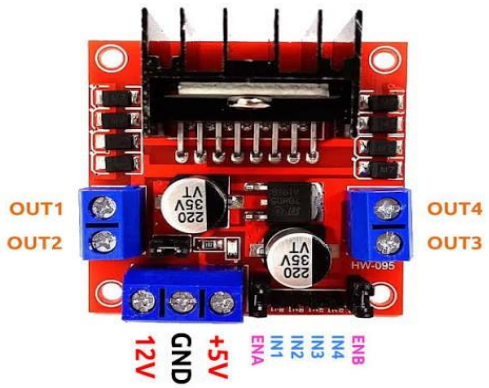

Figure 2 L298N module

DC Gear Motor with Quadrature Encoder (462 PPR)

The actuator is a 12 V DC gear motor with a built-in optical quadrature encoder, salvaged and repurposed from a printer mechanism, communicating over a 6-pin JST connector. The encoder outputs two square-wave channels (A and B) that are 90 degrees out of phase. By reading both channels, the direction of rotation can be determined in addition to the amount of rotation, and by counting edges on both channels (4x decoding) a high-resolution position estimate is obtained. This encoder was measured/verified to produce 462 pulses per revolution (PPR), which sets the angular resolution used to convert raw encoder counts into a real-world shaft angle in degrees.

Figure 3 l motor with encoder and gear

Potentiometers (Setpoint and Live PID Tuning)

Four 10 kOhm potentiometers are connected to ESP32 ADC channels. One potentiometer is used as a manual setpoint dial, letting the user physically set a target angle at the hardware level. The remaining three potentiometers are mapped in firmware to the proportional (Kp), integral (Ki), and derivative (Kd) gains of the PID controller, so the gains can be tuned live, directly at the hardware, while observing the

Figure 4 - potentiometer

I2C LCD Display (16x2, PCF8574 Backpack)

A 16x2 character LCD fitted with a PCF8574 I2C backpack is used for local, on-device feedback. Using I2C instead of a parallel interface reduces the wiring to just two data lines (SDA/SCL) plus power. The LCD continuously displays the live shaft angle, the current target angle, and the active PID gains, allowing the system to be monitored even when it is not connected to MATLAB.

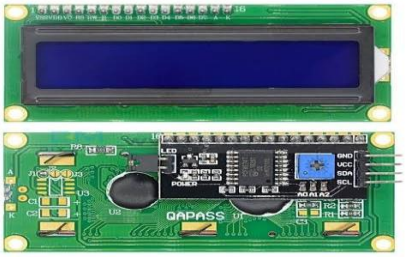

Figure 5 LCD module

PID Control Theory

Proportional-Integral-Derivative (PID) control computes a correction signal u(t) from the error e(t) between the target angle and the measured (live) angle: u(t) = Kp.e(t) + Ki.integral(e(t)) + Kd.(de(t)/dt). The proportional term drives a correction proportional to the present error; the integral term accumulates past error to eliminate steady-state offset; the derivative term reacts to the rate of change of error to dampen overshoot. In this project the PID output is converted into a signed PWM duty cycle sent to the L298N, whose sign sets rotation direction and whose magnitude sets rotation speed, closing the position control loop around the encoder feedback.

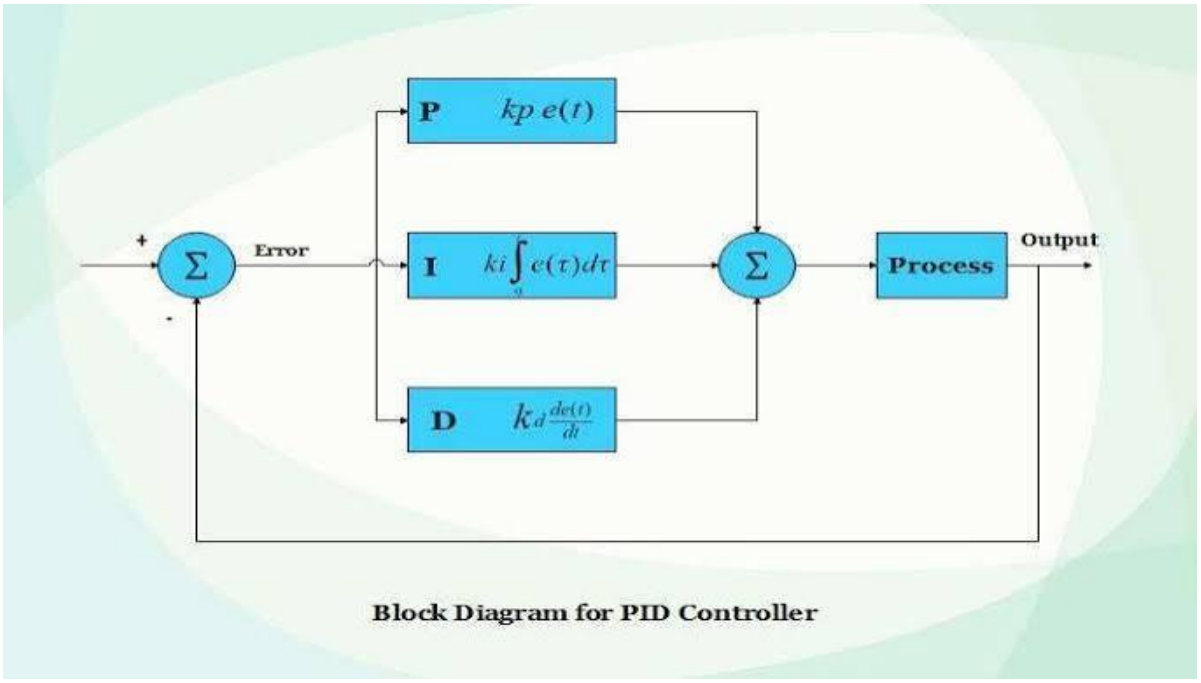

Figure 6 - block diagram for PID controller

Project Block Diagram

The diagram below shows how all subsystems are interconnected: the potentiometers and ESP32 form the control input side, the ESP32 and L298N form the actuation path to the motor, the encoder closes the feedback loop back to the ESP32, the LCD provides local feedback, and the serial link ties the whole system to the MATLAB live bidirectional dashboard for remote monitoring and tuning.

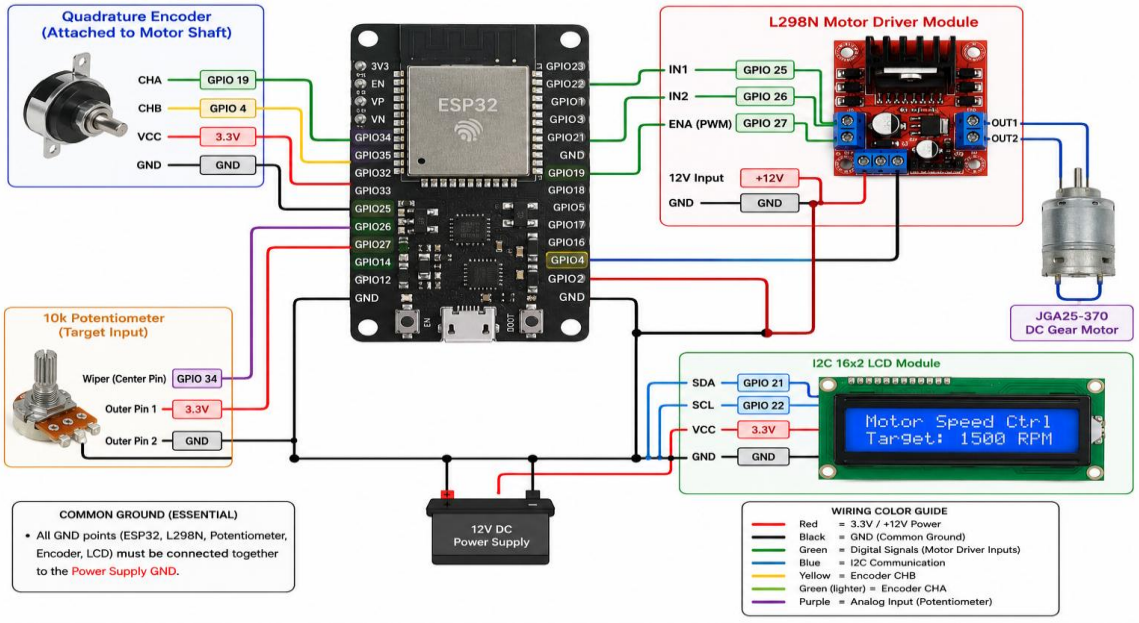

Figure 7 - Overall system block diagram.

Procedure

1. Preliminary System Setup

- Prepare the workspace and ensure the necessary tools (soldering iron, jumper wires, breadboard/PCB) are available.
- Verify the electrical specifications of all components: the JGA25-370 motor (12V), the ESP32 (3.3V logic), and the L298N driver.

2. Hardware Integration and Wiring

- Securely mount the motor and the quadrature encoder onto the mechanical chassis to ensure there is no shaft misalignment.
- Connect the quadrature encoder outputs, Channel A (CHA) to GPIO 19 and Channel B (CHB) to GPIO 4.
- Interface the L298N motor driver control pins: IN1 to GPIO 25, IN2 to GPIO 26, and the ENA pin to GPIO 27 for PWM speed control.
- Wire the center wiper pin of the 10k potentiometer to GPIO 34 to serve as the user-controlled setpoint input.
- Connect the I2C 16x2 LCD module SDA pin to GPIO 21 and the SCL pin to GPIO 22.
- Establish a **Common Ground** by connecting all negative terminals (Power Supply, ESP32, L298N, Encoder, and LCD) to a single ground point to eliminate electrical noise.

3. Firmware Implementation

- Write the control algorithm in the Arduino IDE.
- Implement an Interrupt Service Routine (ISR) for the encoder, using the `IRAM_ATTR` attribute to ensure high-priority pulse counting even during high-speed motor rotation.
- Program the PID logic to continuously calculate the error between the potentiometer target and the current encoder position.
- Configure the Serial communication at 115200 baud to allow for bidirectional data exchange with the MATLAB environment.
- Flash the code to the ESP32 and use the Serial Monitor to verify that the pulse count increases/decreases correctly when the motor shaft is rotated.

4. MATLAB Dashboard & Interface Setup

- Launch MATLAB and initialize a serial object to communicate with the COM port assigned to the ESP32.
- Develop a GUI (Graphical User Interface) containing three sliders for real-time manual adjustment of K_p, K_i, and K_d PID parameters.
- Create a live data plotting window within the GUI to visualize the "Target Position" vs. "Actual Motor Position" in real-time.

5. PID Tuning and Operational Calibration

- Power the motor driver with the external 12V DC source, ensuring the ESP32 is powered separately via USB.
- Observe the "Steady-State Error"—a 21^\circ offset caused by mechanical friction within the gear train.
- Gradually increase the Integral gain (K_i) using the MATLAB sliders to provide enough force to overcome the friction and reduce the steady-state error to 0^\circ.
- Adjust the Proportional gain (K_p) to ensure the motor reaches the target set point quickly.

- Fine-tune the Derivative gain (K_d) to dampen the system response and stop the motor from oscillating or overshooting the setpoint.

6. Final Performance Validation

- Perform a sweep test by rotating the potentiometer from minimum to maximum range to ensure the motor follows the input smoothly and accurately.
- Confirm that the 16x2 LCD screen updates the position display in real-time alongside the MATLAB dashboard visualization.
- Test the system stability under load to ensure that the PID gains remain effective and the system does not lose its position calibration.

MATLAB Live Bidirectional Control Dashboard Development:

A MATLAB application was developed to communicate with the ESP32 over serial in both directions. On the receiving side, the dashboard plots the live shaft angle against the target angle in real time and displays the currently active Kp, Ki, and Kd values reported by the firmware. On the transmitting side, the dashboard exposes interactive sliders for Kp, Ki, and Kd (allowing the user to override the potentiometer-set gains remotely) and accepts a numeric target angle, which is sent back to the ESP32 to update the setpoint - making the tuning loop fully bidirectional between the hardware and MATLAB.

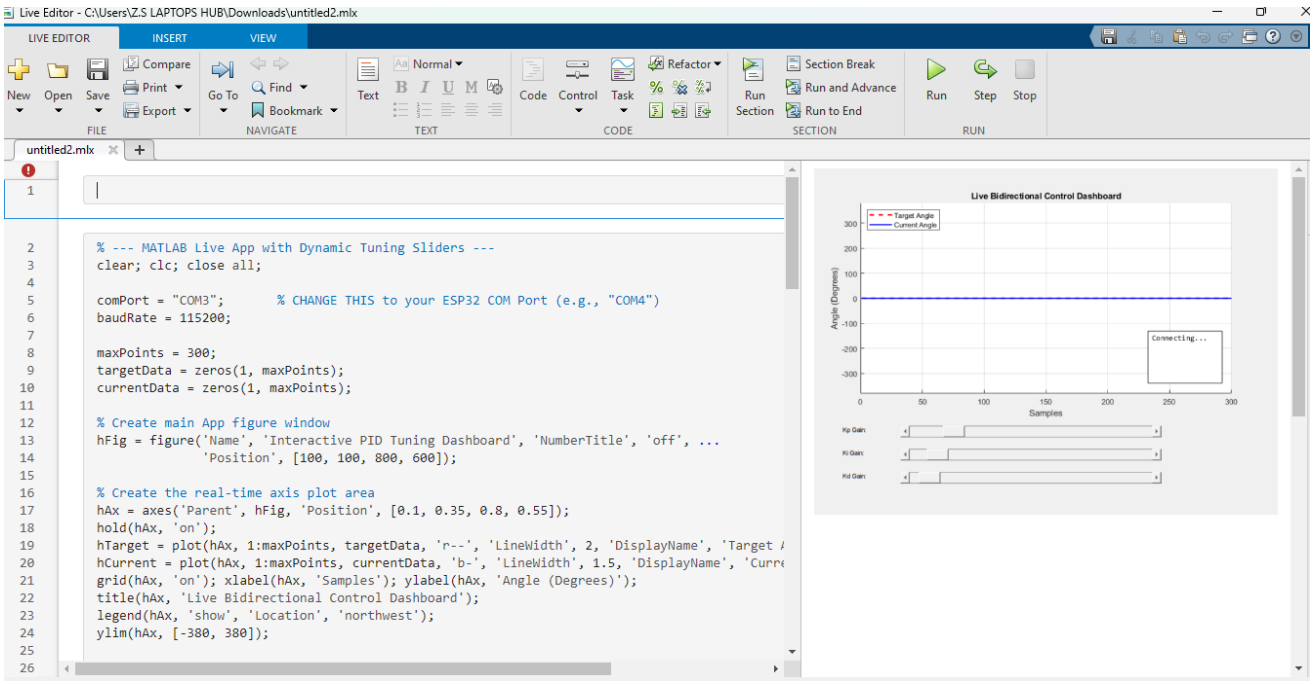

Figure - MATLAB Live Bidirectional Control

System Integration and Testing

With firmware and dashboard both complete, the full system was assembled and powered up together: ESP32 connected to the PC over USB for the MATLAB link, motor and encoder wired through the L298N, and potentiometers/LCD active for local control and feedback. A series of step-response trials were then run to tune the PID gains, described next in the Results section.

Results

MATLAB Live Bidirectional Dashboard

The completed MATLAB dashboard provided a real-time plot of live angle versus target angle for every trial, together with an on-screen readout of the exact target angle, the settled output angle, and the active Kp/Ki/Kd values at that moment. This let the PID gains be adjusted - either from the physical potentiometers or directly from the MATLAB sliders - while immediately observing the effect on the step response, which was the basis for the tuning trials below.

PID Tuning Trials

Representative dashboard captures from the tuning trials are shown below.

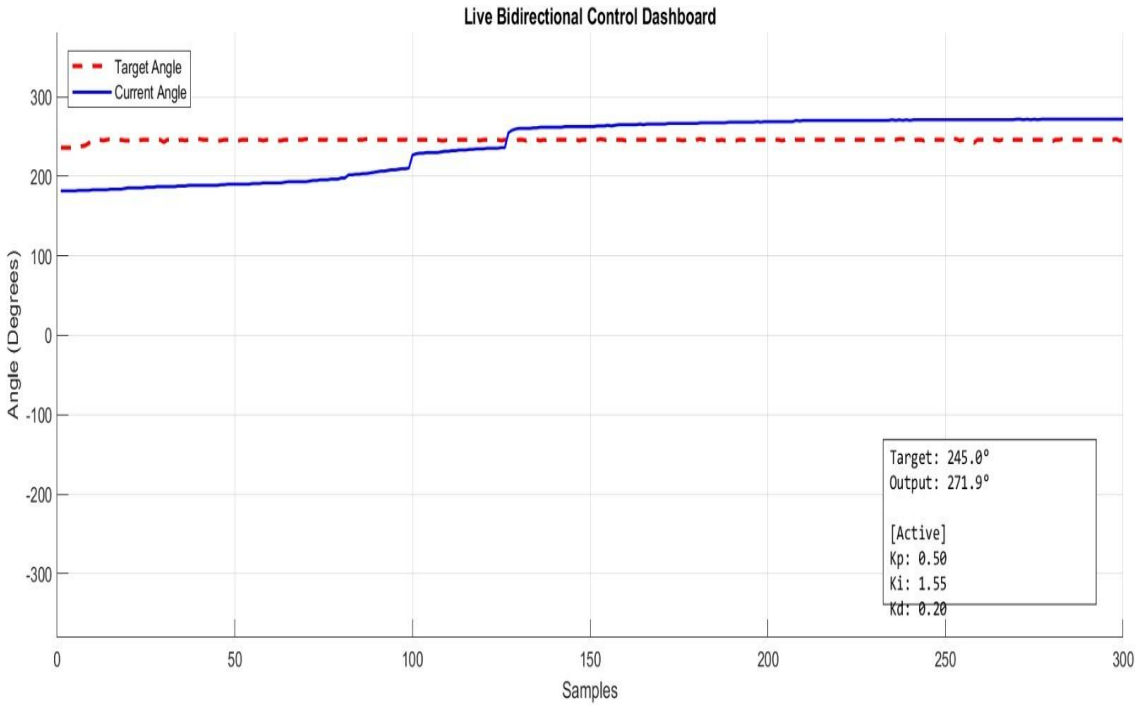

Figure 8.1 - Trial A: Kp = 0.50, Ki = 1.55, Kd = 0.20. Target 245.0 deg, settled output 271.9 deg. The high integral gain relative to Kp caused the response to overshoot the target and settle above it, indicating Ki was too aggressive for this loading condition.

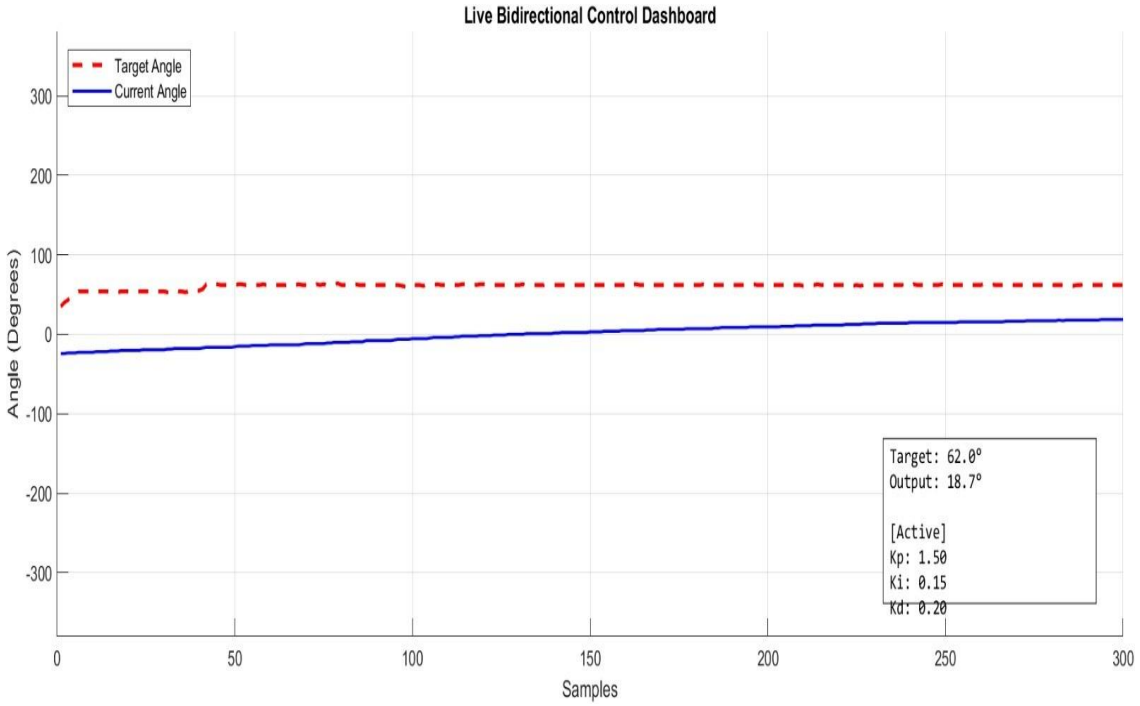

Figure 8.2 - Trial B: Kp = 1.50, Ki = 0.15, Kd = 0.20. Target 62.0 deg, output only reached 18.7 deg within the sample window. The response is sluggish, showing that Kp alone was not sufficient to drive the shaft to target quickly with this low integral action.

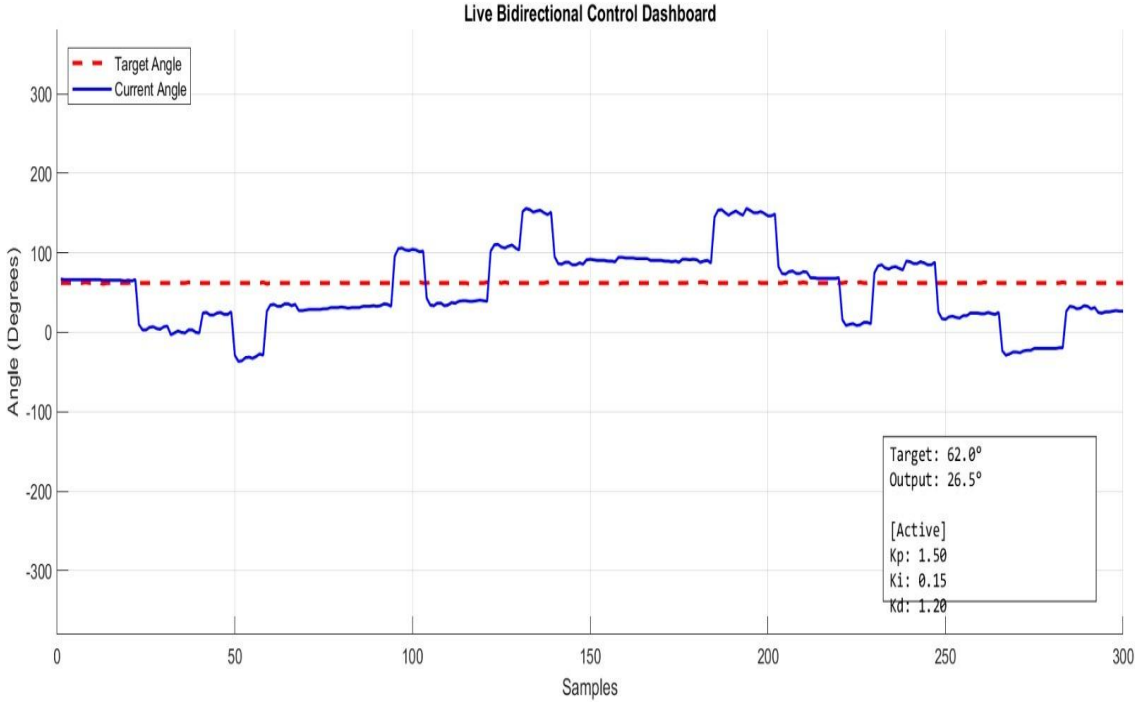

Figure 8.3 - Trial C: Kp = 1.50, Ki = 0.15, Kd = 1.20. Target 62.0 deg, output 26.5 deg. Raising Kd introduced noticeable oscillation and overshoot around the target band instead of a smooth approach, showing the derivative gain was amplifying encoder noise/disturbance rather than damping the response.

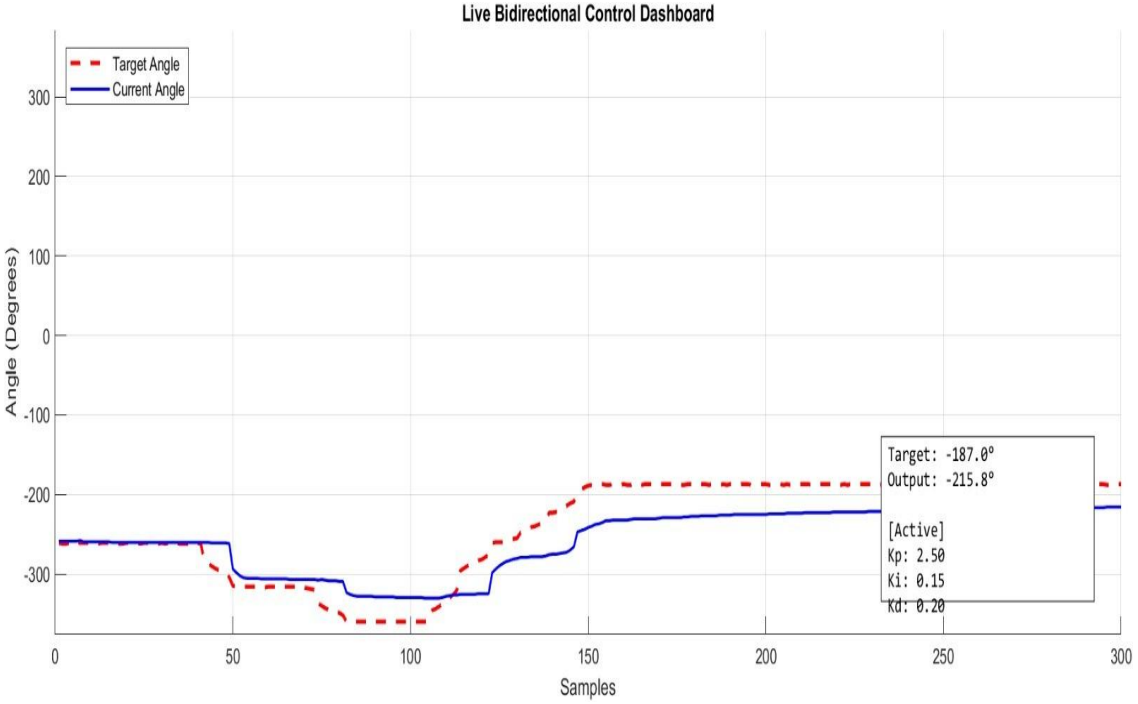

Figure 8.4 - Trial D: Kp = 2.50, Ki = 0.15, Kd = 0.20. Target -187.0 deg, output -215.8 deg, showing correct bidirectional tracking into negative angles as the target step changed direction, with the live angle following the target trajectory closely.

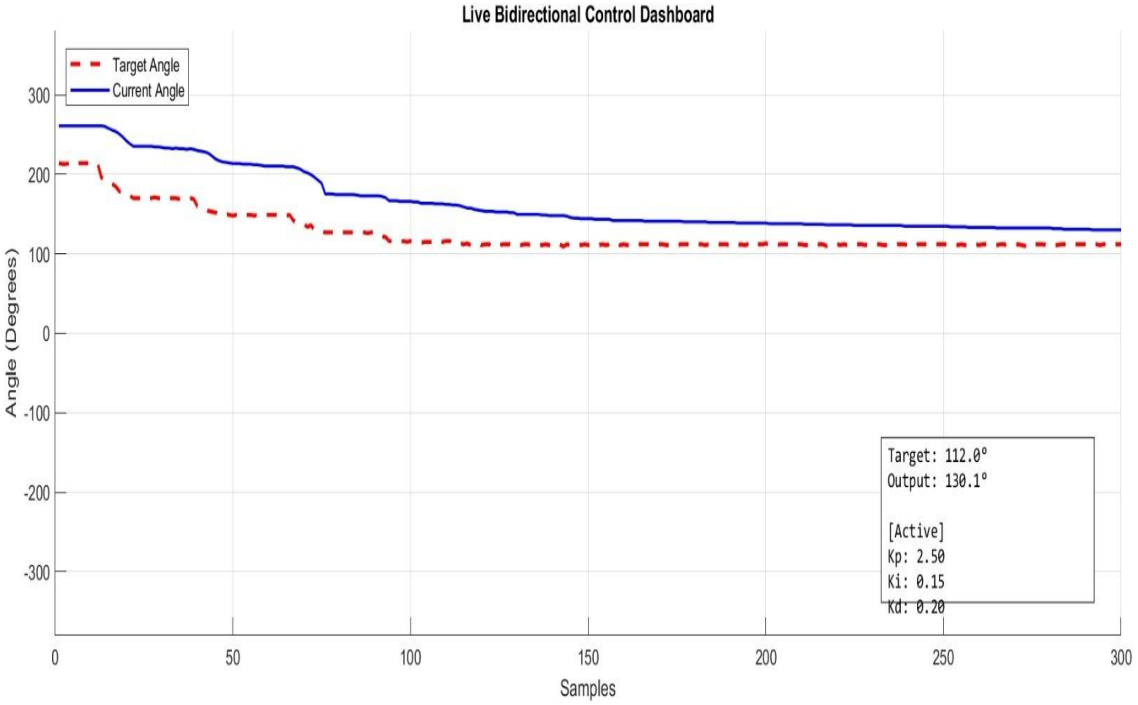

Figure 8.5 - Trial E: Kp = 2.50, Ki = 0.15, Kd = 0.20. Target 112.0 deg, output 130.1 deg, converging smoothly toward the setpoint with reduced overshoot compared to earlier trials, confirming this gain combination as the best-performing set found during tuning.

1.1 Summary of Tuning Results

Trial	Kp	Ki	Kd	Target (deg)	Output (deg)	Observation
A	0.50	1.55	0.20	245.0	271.9	Overshoot / high Ki
B	1.50	0.15	0.20	62.0	18.7	Sluggish / low Ki
C	1.50	0.15	1.20	62.0	26.5	Oscillatory / high Kd
D	2.50	0.15	0.20	-187.0	-215.8	Good bidirectional tracking
E	2.50	0.15	0.20	112.0	130.1	Best overall response

Across the five tuning trials, a clear pattern emerged. High integral gain relative to proportional gain (Trial A) produced overshoot and settling above the target. Low integral gain with moderate proportional gain (Trial B) produced a sluggish, slow-converging response. Increasing derivative gain without addressing the underlying proportional/integral balance (Trial C) introduced oscillation rather than damping. The best overall performance was achieved with Kp = 2.50, Ki = 0.15, Kd = 0.20 (Trials D and E), which tracked both positive and negative target steps with reasonable speed and acceptable overshoot, confirming correct bidirectional control of the motor position. These final tuned gains were adopted as the operating configuration for the system, with the option to fine-tune further through either the on-board potentiometers or the MATLAB dashboard sliders depending on load conditions.

2. Conclusion

This project successfully implemented a dual-mode adaptive PID position control system for a DC gear motor using an ESP32, L298N motor driver, and a 462 PPR quadrature encoder, with live gain and setpoint control available both locally through potentiometers and remotely through a custom MATLAB bidirectional dashboard. Systematic tuning trials identified Kp = 2.50, Ki = 0.15, Kd = 0.20 as the best-performing gain set, achieving reliable bidirectional angle tracking with acceptable overshoot and settling behaviour. The combination of on-device feedback (LCD), local tuning (potentiometers), and remote live monitoring/tuning (MATLAB dashboard) demonstrates a complete, testable closed-loop control system suitable for further extension, such as automated gain scheduling or load-adaptive retuning.